HOCHSCHULE ESSLINGEN

OBJEKTORIENTIERTE SYSTEME

BESONDERE KLASSENELEMENTE IN C++

PROF. DR. JÖRG NITZSCHE
HOCHSCHULE ESSLINGEN
JOERG.NITZSCHE@HS-ESSLINGEN.DE

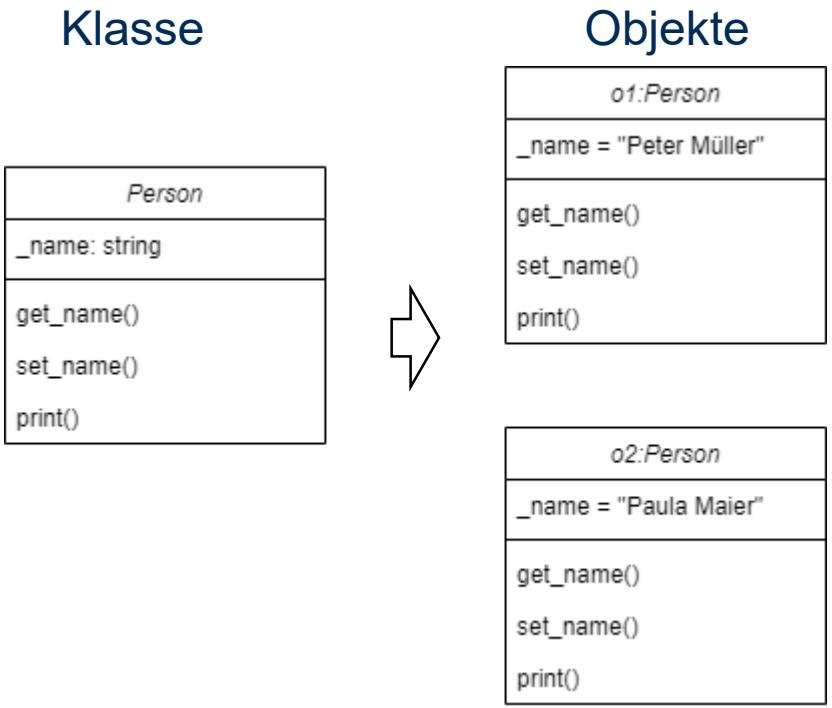
LERNZIEL

- Besondere Klassenelemente in C++ kennenlernen und zur Problemlösung anwenden können
  - static
  - const
- Funktionsweise von Freundfunktionen und Freundklassen verstehen

HOCHSCHULE ESSLINGEN

1 Klassen und Objekte
2 Statische Datenelemente
3 Statische Funktionen
4 Konstante Datenelemente in Objekten
5 Konstante Objekte
6 Freundfunktionen
7 Freundklassen

KONZEPT EINER KLASSE



- Klasse = Schema zur Erzeugung von Objekten
- alle Objekte (Instanzen) einer Klasse haben die gleichen Eigenschaften (Attribute) und Fähigkeiten (Methoden)

INTERNE REPRESENTATION VON KLASSEN UND METHODEN

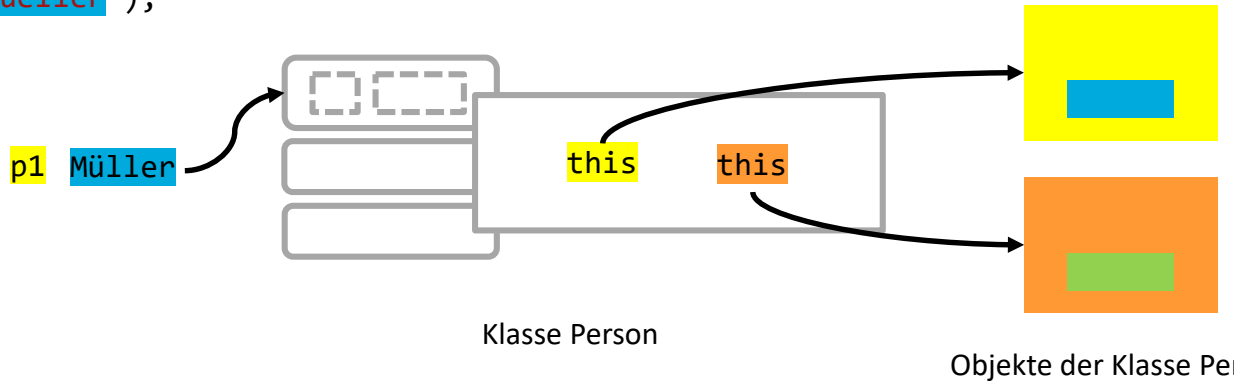
- Die Methoden einer Klasse werden nur einmal im Speicher abgelegt. Die einzelnen Instanzen liegen ebenfalls im Speicher, enthalten aber nur die Daten, also die Attribute.
- Wird eine Methode zu einer speziellen Instanz aufgerufen, dann braucht diese Methode die Information, wo die Instanzattribute sich befinden.
- Diese Information stellt der Compiler im Zeiger **`this`** zur Verfügung, der innerhalb einer Methode auch explizit verwendet werden kann.
- Jeder Methode wird bei ihrem Aufruf der **`this`**-Zeiger übergeben. Er zeigt auf das aufrufende Objekt der Klasse, zu der die Methode gehört.

METHODEN UND THIS

```
Person p1;
p1.set_name("Mueller");
```

Es wird intern die Methode setName der Klasse Person aufgerufen. p1 wird als this Zeiger der Methode übergeben:

```
Person.set_name(p1, "Mueller");
```



HOCHSCHULE ESSLINGEN

1 Klassen und Objekte

2 Statische Datenelemente

3 Statische Funktionen

4 Konstante Datenelemente in Objekten

5 Konstante Objekte

6 Freundfunktionen

7 Freundklassen

STATISCHE KLASSENELEMENTE

- Ausgangssituation:
  - Instanz = Objekt
  - Instanzmethode = Objektmethode
  - Klassen definieren Instanzvariablen und Instanzmethoden
  - Jedes Objekt einer Klasse besitzt eigene Instanzvariablen und nutzt die Instanzmethoden der Klasse. D.h. zwei Objekte einer Klasse haben Variablen mit den gleichen Namen, die aber unterschiedliche Speicherbereiche belegen.
- Manchmal möchte man Daten über die Menge aller Objekte einer Klasse speichern. Dazu dienen dann:
  → statische Datenelemente, statische Methoden

BEISPIEL TAGEGELDKONTO (1)

- Folgende Vorstellung: Eine Klasse **CallMoneyAccount (TagegeldKonto)** enthält den Kontostand und den aktuellen Zinssatz. Jede Instanz entspricht einem Sparkonto.
- Ändert sich der Zinssatz, muss die Änderung mühsam in jeder Instanz durchgeführt werden.
- Der Einsatz einer globalen Variablen kommt wegen des Prinzips der Datenkapselung nicht in Frage.
- Die Lösung ist eine Variable, die pro Klasse nur einmal vorhanden ist. Diese Variable ist in jeder Instanz sichtbar, kann aber nach außen gekapselt werden (private!).
- Diese Art von Variable nennt sich **static**-Attribut/Variable oder auch Klassenattribut/Klassenvariable.

BEISPIEL TAGEGELDKONTO (2)

```
#include<string>
using namespace std;

class CallMoneyAccount
{
    string _customer;
    float _balance;
    static float _s_rate; //Klassenattribut
public:
    CallMoneyAccount();
    void set_customer(string customer) {
        _customer = customer;
    }
    void pay_in(float payment) {
        _balance = _balance + payment;
    }
    void add_rate() {
        _balance = _balance * (1 + _s_rate);
    }
};
```

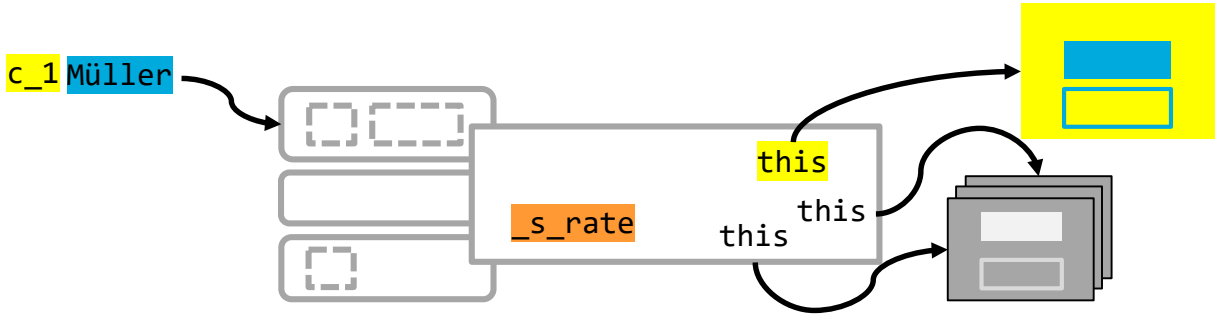
```
float CallMoneyAccount::_s_rate = 2.0;
int main() {

    CallMoneyAccount c_1;
    c_1.set_customer("Mueller");
    c_1.pay_in(135.95);
    c_1.add_rate();

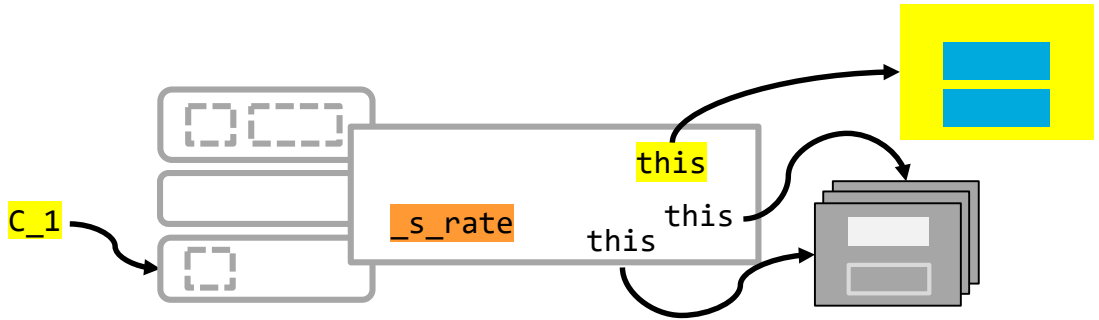
    return 0;
}
```

KLASSENATTRIBUT

```
CallMoneyAccount c_1;
c_1.set_costumer("Mueller");
```




```
...
c_1.add_rate()
```




```
_balance = _balance * (1 + _s_rate);
```

BEISPIEL FÜR KLASSENVARIABLEN

```
1  class Student
2  {
3      int _student_nr;
4      static int _s_student_count;
5
6  public:
7      Student()
8      {
9          _student_nr = ++_s_student_count;
10     // _student_count wird bei
11     // jedem neuen Student erhöht
12     }
13     void count()
14     {
15         cout << "Ich bin die Nummer "
16              << _student_nr << endl;
17     }
18 };
```

```
19 // Initialisierung ausserhalb der Klasse!
20 int Student::_s_student_count = 0;
21 void main()
22 {
23     array<Student,10> semester;
24     // 10 Objekte der Klasse Student
25     // instantiiert
26     for (int i = 0; i < 10; i++)
27     {
28         semester[i].count();
29     }
30 }
```

HOCHSCHULE ESSLINGEN

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Particify - QUIZ

```
#include<iostream>
using namespace std;
class Student {
    int _student_nr;
    static int _s_student_count;
public:
    Student() {
        _student_nr = ++_s_student_count;
    }
    ~Student() {
        _s_student_count--;
    }
    void print() {
        cout << "Ich bin die Nummer "
             << _student_nr << endl;
    }
};
```

```
int Student::_s_student_count = 0;
int main()
{
    Student s_1;
    Student s_2;
    Student s_3;
    s_2.~Student();
    Student s_4;
    s_1.print();
    s_3.print();
    s_4.print();

    return 0;
}
```

- Was läuft aus Ihrer Sicht hier falsch?

Qrticify - QUIZ: AUFLÖSUNG


```
1  #include<iostream>
2  using namespace std;
3  class Student {
4      int _student_nr;
5      static int _s_student_count;
6      static int _s_id_generator;
7  public:
8      Student() {
9          _student_nr = ++_s_id_generator;
10         _s_student_count++
11     }
12     ~Student() {
13         _s_student_count--;
14     }
15     void print()
16     {
17         cout << "Ich bin die Nummer "
18              << _student_nr << endl;
19     }
20 };
```

- Klassenvariablen richtig verwenden

```
21 int Student::_s_student_count = 0;
22 int Student::_s_id_generator = 0;
23 int main()
24 {
25     Student s_1;
26     Student s_2;
27     Student s_3;
28     s_2.~Student();
29     Student s_4;
30     s_1.print();
31     s_3.print();
32     s_4.print();
33
34     return 0;
35 }
```

KLASSENVARIABLEN

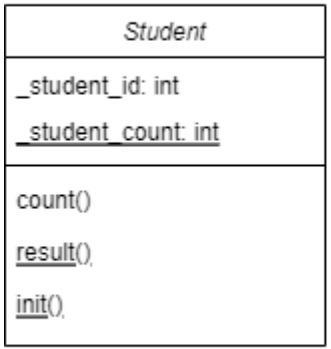
- Statische Datenelemente (= Klassenvariablen) beschreiben Eigenschaften einer Klasse (die Menge aller Objekte) und nicht Eigenschaften von einzelnen Objekten.
- Sie verhalten sich wie globale Variablen, auf die jedes Klassenobjekt zugreifen kann.
- Sie werden in der Klasse deklariert und außerhalb unter zuhilfenahme des Scope Operators initialisiert
- Sie können benutzt werden, auch wenn es noch kein Objekt der Klasse gibt.

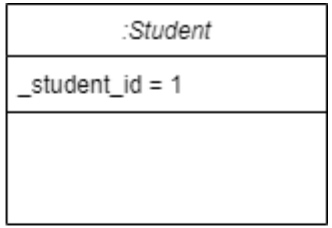
HOCHSCHULE ESSLINGEN

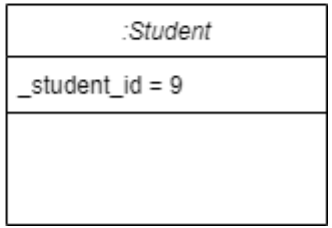
1 Klassen und Objekte
2 Statische Datenelemente
3 Statische Funktionen
4 Konstante Datenelemente in Objekten
5 Konstante Objekte
6 Freundfunktionen
7 Freundklassen

STATISCHE FUNKTIONEN

- Statische Methoden dürfen nur auf statische Datenelemente zugreifen.
- Sie können daher ohne Objekt aufgerufen werden. Sie verfügen über keinen this-Zeiger.







BEISPIEL: STATISCHE METHODEN (1)

```
#include<iostream>
using namespace std;
class Student {
    int _student_id;
    static int _s_student_count;
    static int _s_id_generator;
public:
    Student() { _student_id = ++_s_id_generator; ++_s_student_count; }
    ~Student() { _s_student_count--; }
    void count()
    { // Instanzmethode
        cout << "Ich bin die Nummer " << _student_id << endl;
    }
    static void result()
    { // statische Methode
        cout << "Schuelerzahl: " << _s_student_count << endl;
    }
    static void init()
    { // statische Methode
        _s_student_count = 0; _s_id_generator = 0;
    }
};
```

BEISPIEL: STATISCHE METHODEN (2)

```
1  int Student::_s_student_count;// Deklaration notwendig!!!
2  int Student::_s_id_generator; // Deklaration notwendig!!!
3  void main()
4  {
5      Student::init();       // Initialisierung hier
6      Student::result();
7      Student school_class[10];    // jetzt Objekte vorhand
8
9      // schueler[1].init();  // Init auch hier möglich
10
11     for (int i = 0; i < 10; i++)
12         school_class[i].count(); // Instanzmethode
13
14     for (int i = 0; i < 10; i++)
15         school_class[i].result();  // statische Methode
16 }
```

Ausgabe

```
Schuelerzahl: 0
Ich bin die Nummer 1
Ich bin die Nummer 2
Ich bin die Nummer 3
Ich bin die Nummer 4
Ich bin die Nummer 5
Ich bin die Nummer 6
Ich bin die Nummer 7
Ich bin die Nummer 8
Ich bin die Nummer 9
Ich bin die Nummer 10
Schuelerzahl: 10
Schuelerzahl: 10
Schuelerzahl: 10
Schuelerzahl: 10
Schuelerzahl: 10
Schuelerzahl: 10
Schuelerzahl: 10
Schuelerzahl: 10
Schuelerzahl: 10
Schuelerzahl: 10
```

STATISCHE METHODEN

- Statische Methoden dürfen nur auf statische Datenelemente zugreifen.
- Sie verfügen über keinen this-Zeiger.
- Sie können aufgerufen werden, auch wenn es (noch) kein Objekt der Klasse gibt.
- Beim Aufruf einer statischen Methode stellt man den Klassennamen mit dem Scope-Operator vor den Namen der Methode.

ZUGRIFF AUF DATENELEMENTE

- Auf statische Datenelemente können zugreifen:
  - statische Methoden
  - Instanzmethoden
  - Freundfunktionen (kommen später dran)
- Auf nichtstatische Datenelemente können nur nicht-statische ("normale") Methoden zugreifen (this-Zeiger).
- Eine statische Methode oder eine Freundfunktion kann auf nichtstatische Datenelemente eines Objektes der Klasse zugreifen, das man ihr als Parameter übergeben hat.

BEISPIEL: ZUGRIFF AUF NICHTSTATISCHE DATENELEMENTE

```
1  class Student
2  {
3      int _student_id;
4      static int _s_student_count;
5  public:
6      Student() { ... }
7      void count() const { ... }
8      static void result() { ... }
9      static void init() { ... }
10     static void print(const Student &s)
11     {                                          // statische Methode, der
12         cout << _s_student_count << endl;      // Objekt übergeben wird.
13         cout << s._student_id << endl;         // Zugriff auf Instanz-
14     }                                          // variable wird möglich.
15 };
16 int Student::_s_student_count = 0;
17 void main() {
18     Student school_class[10];
19     Student::print(school_class[5]);
20 }
```

STATISCHE VS. NICHTSTATISCHE METHODEN

- Statische Methoden werden immer statisch gebunden. Es gibt damit beim Aufruf keinen Overhead für dynamisches Binden
  Beim Aufruf einer statischen Methode entfällt die Verwaltung des this-Zeigers.
- Will man zusammengehörende globale Funktionen zusammenfassen, kann man sie als statische Methoden einer Klasse realisieren:

```
class Math { public:
  static double sin(double x);
  static double cos(double x);
  ……
```

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Particify - QUIZ

```
#include<string>
#include<iostream>
using namespace std;

class CallMoneyAccount {
    float balance;
    static float rate; //Klassenattribut
public:
    CallMoneyAccount(){};
    void pay_in(float payment) {
        balance = balance + payment;
    }
    void add_rate() {
        balance = balance * (1 + rate);
    }
    static void set_rate(float rate) {
        rate = rate;
    }
    static float get_rate() {
        return rate;
    }
};
```

```
float CallMoneyAccount::rate = 2.0;
int main() {

    CallMoneyAccount c_1;
    c_1.pay_in(135.95);
    CallMoneyAccount::set_rate(2.5);
    c_1.add_rate();
    cout << CallMoneyAccount::get_rate() << endl;

    return 0;
}
```

- Was wird hier auf der Konsole ausgegeben?

Quizify - QUIZ: AUFLÖSUNG

```cpp
#include<string>
#include<iostream>
using namespace std;

class CallMoneyAccount {
    float balance;
    static float rate; //Klassenattribut
public:
    CallMoneyAccount(){};
    void pay_in(float payment) {
        balance = balance + payment;
    }
    void add_rate() {
        balance = balance * (1 + rate);
    }
    static void set_rate(float rate) {
        rate = rate;
    }
    static float get_rate() {
        return rate;
    }
};
```

```cpp
float CallMoneyAccount::rate = 2.0;
int main() {

    CallMoneyAccount c_1;
    c_1.pay_in(135.95);
    CallMoneyAccount::set_rate(2.5);
    c_1.add_rate();
    cout << CallMoneyAccount::get_rate() << endl;

    return 0;
}
```

- 2
- Set_rate() greift nicht auf die statische Variable zu

NAMENSKOLLISIONEN

```
#include<string>
#include<iostream>
using namespace std;

class CallMoneyAccount {
    float balance;
    static float rate; //Klassenattribut
public:
    CallMoneyAccount(){};
    void pay_in(float payment) {
        balance = balance + payment;
    }
    void add_rate() {
        balance = balance * (1 + rate);
    }
    static void set_rate(float rate) {
        CallMoneyAccount::rate = rate;
    }
    static float get_rate() {
        return rate;
    }
};
```

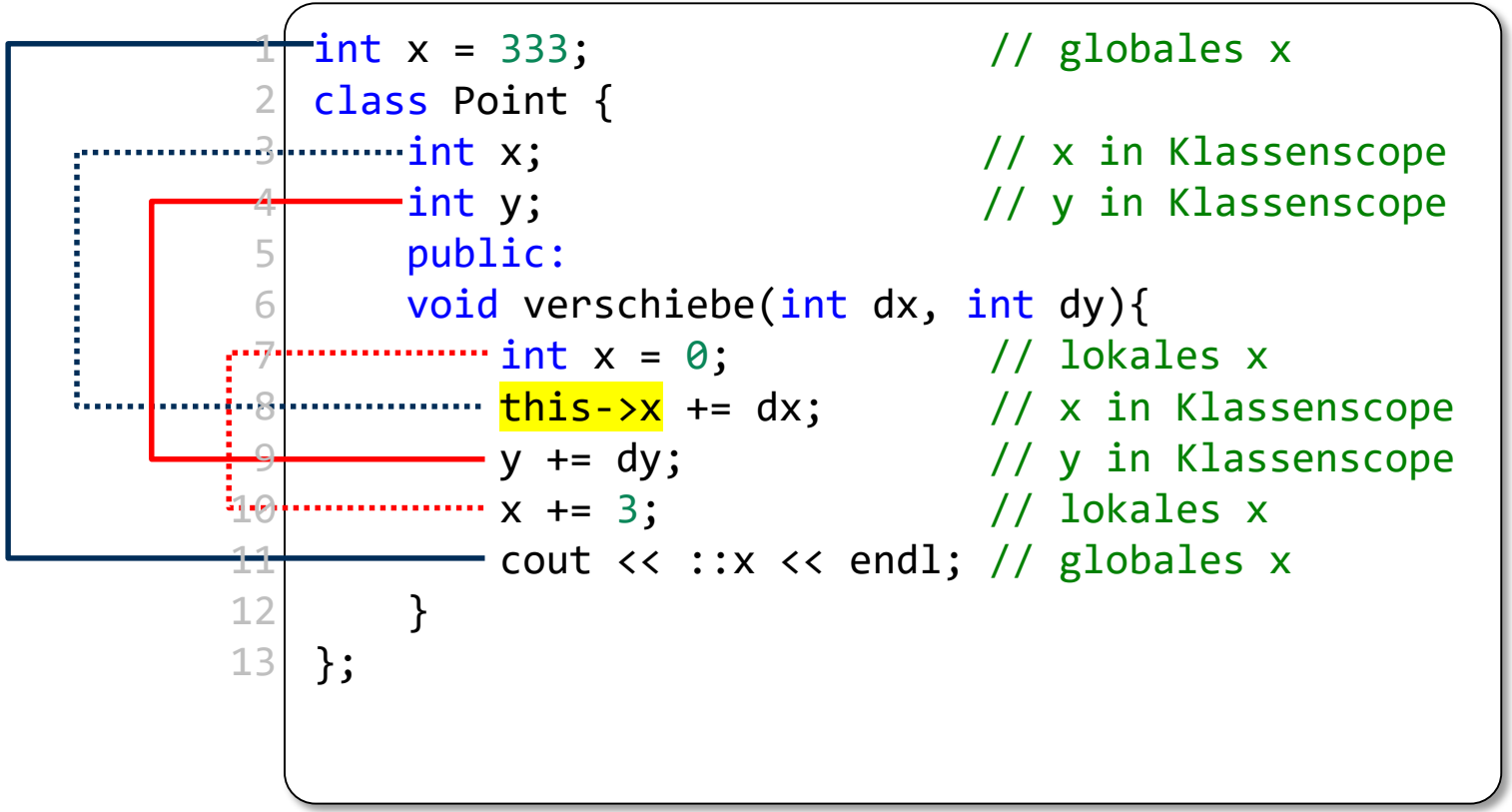
```
float CallMoneyAccount::rate = 2.0;
int main() {

    CallMoneyAccount c_1;
    c_1.pay_in(135.95);
    CallMoneyAccount::set_rate(2.5);
    c_1.add_rate();
    cout << CallMoneyAccount::get_rate() << endl;

    return 0;
}
```

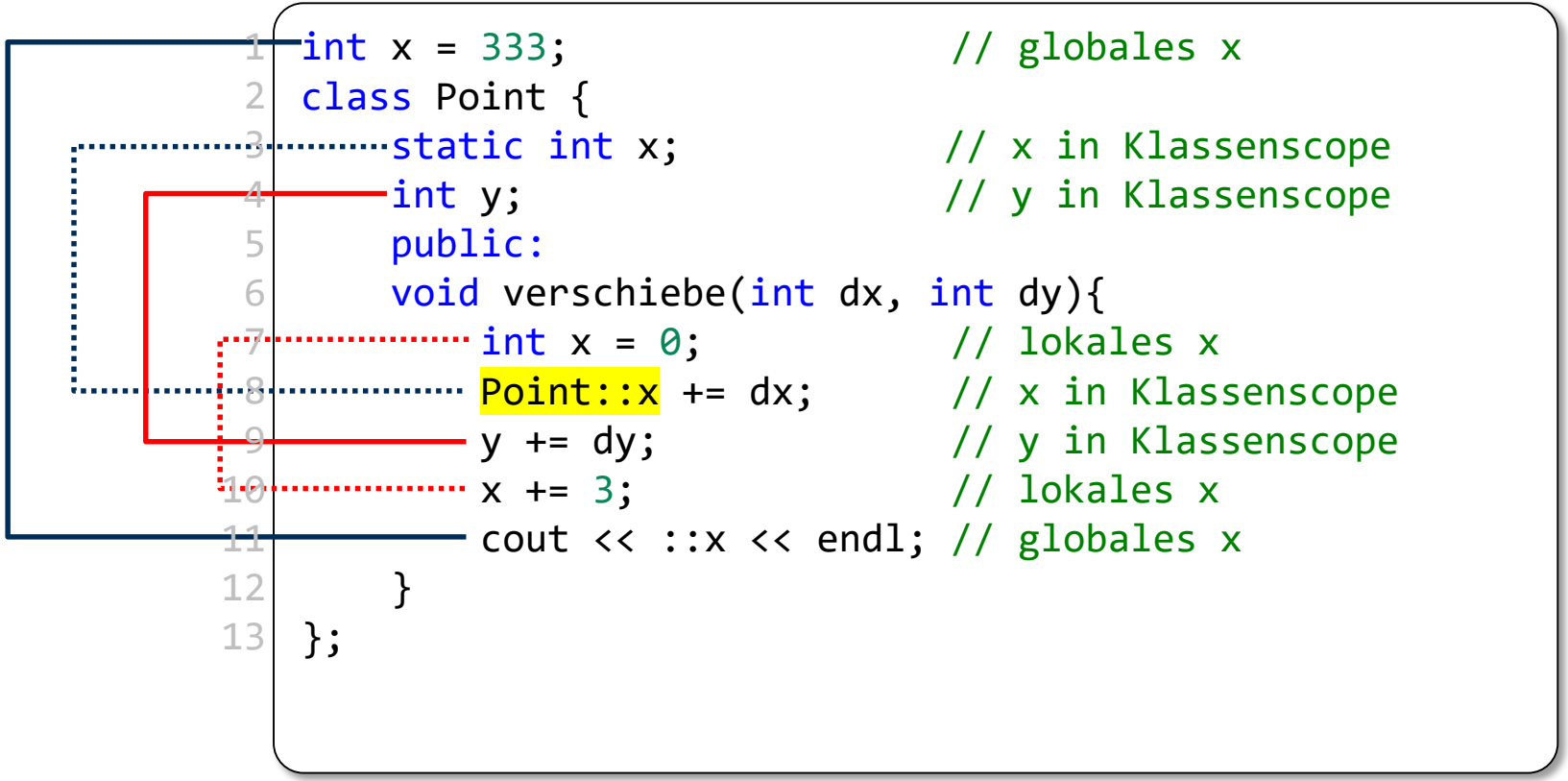
- Bei Namenskollisionen kann der Scope-Operator verwendet werden um auf die Klassenvariable zu zeigen

ZUGRIFF MIT SCOPE-OPERATOR UND THIS



```
1  int x = 333;                        // globales x
2  class Point {
3      int x;                          // x in Klassenscope
4      int y;                          // y in Klassenscope
5      public:
6      void verschiebe(int dx, int dy){
7          int x = 0;                  // lokales x
8          this->x += dx;              // x in Klassenscope
9          y += dy;                    // y in Klassenscope
10         x += 3;                     // lokales x
11         cout << ::x << endl; // globales x
12     }
13 };
```

ZUGRIFF MIT SCOPE-OPERATOR



```
1  int x = 333;                      // globales x
2  class Point {
3      static int x;                 // x in Klassenscope
4      int y;                        // y in Klassenscope
5      public:
6      void verschiebe(int dx, int dy){
7          int x = 0;                // lokales x
8          Point::x += dx;           // x in Klassenscope
9          y += dy;                  // y in Klassenscope
10         x += 3;                   // lokales x
11         cout << ::x << endl; // globales x
12     }
13 };
```

HOCHSCHULE ESSLINGEN

1 Klassen und Objekte
2 Statische Datenelemente
3 Statische Funktionen
4 Konstante Datenelemente in Objekten
5 Konstante Objekte
6 Freundfunktionen
7 Freundklassen

KONSTANTE DATENELEMENTE IN OBJEKTEN

- Konstante Datenelemente können für jedes Objekt verschieden sein.
- Sie werden erst zur Laufzeit festgelegt (z.B. über einen Parameter beim Konstruktor), sind aber ab dann konstant und dürfen nicht mehr geändert werden.
- Die Initialisierung eines konstanten Datenelementes muss über die Initialisierungsliste eines Konstruktors erfolgen.

KONSTANTE DATENELEMENTE IN OBJEKTEN (1)

```
1  class Car
2  {
3    const int _c_vehicle_identification_number;
4    int _tank_contents, _mileage;
5
6  public: // Initialisierungsliste des Konstruktors
7    Car(int vin) : _c_vehicle_identification_number(1234567890), _tank_contents(20)
8    {
9      _mileage = 0; // Anweisungsblock des Konstruktors
10   }
11   void drive(int km)
12   {
13     _mileage += km;
14     _tank_contents = _tank_contents – (km * 0,01);
15   }
16   void repair(int km)
17   {
18     _c_vehicle_identification_number = 1234567891; // Fehler !
19   }
20 };
```

HOCHSCHULE ESSLINGEN

1 Klassen und Objekte
2 Statische Datenelemente
3 Statische Funktionen
4 Konstante Datenelemente in Objekten
5 Konstante Objekte
6 Freundfunktionen
7 Freundklassen

KONSTANTE OBJEKTE

```
1  class Fraction {
2    int _numerator, _denominator;
3  public:
4    Fraction (int _numerator =0 , int _denominator =1);
5    Fraction (const Fraction& f);
6    int get_numerator();
7    int get_denominator();
8    void print();
9  };
10
11 int main() {
12   const Fraction f(3,5);
13   f.get_numerator();  // Fehler: get_numerator() nicht const
14   f.print();          // Fehler: print() nicht const
15 }
16
```

- Objekte können selbst als konstant definiert sein.
- Auf konstanten Objekten können keine normalen Methoden aufgerufen werden

- Ein weiteres Beispiel für konstante Objekte sind mit **const** geschützte Parameter:

```
Fraction::Fraction(const Fraction& f)
{
    cout << "Kopierkonstruktor" << endl;

    _numerator = b.get_numerator();
    _denominator = b.get_denominator();
}
```

KONSTANTE INSTANZMETHODEN (1)

- Eine Instanzmethode, die für ein konstantes Objekt aufgerufen werden soll, muss mit **`const`** zwischen Parameterliste und Funktionsblock deklariert werden:

```
class Fraction
{
    int _numerator, _denominator;

public:
    int get_numerator() const;
    void print() const;
    void set_numerator(int z);
    // darf natürlich nicht const sein, da Objekt geändert wird.
    ...
};
```

KONSTANTE INSTANZMETHODEN (2)

- Das Schlüsselwort **`const`** muss auch bei der Definition der Methode stehen:

```
int Fraction::get_numerator() const {
    return _numerator;
}
void Fraction::set_numerator(int n) const {
    _numerator = n; // Fehlermeldung da const
}
```

- Eine mit **`const`** definierte Methode darf weder Datenelemente des this-Objekts verändern noch nicht-const Funktionen für das this-Objekt aufrufen.

KONSTANTE INSTANZMETHODEN (3)

- Methoden, die auf einem **konstanten** Objekt ausgeführt werden, müssen als **const** deklariert sein.
- Methoden, die auf einem **nicht-konstanten** Objekt ausgeführt werden, **können** als **const** deklariert sein.
- Ist sowohl eine const- als auch eine nicht-const-Methode deklariert, so wird bei einem nicht-konstanten Objekt die nicht-const-Methode ausgeführt, bei einem const-Objekt hingegen die const-Methode.

KONSTANTE INSTANZMETHODEN (4) - BEISPIEL

```
1  #include<iostream>
2  using namespace std;
3
4  class Class {
5  public:
6      void print() {
7          cout << "Eine nicht-konstante Methode" << endl;
8      }
9      void print() const {
10         cout << "Eine konstante Methode" << endl;
11     }
12 };
13 int main() {
14
15     Class not_constant;
16     const Class constant;
17     not_constant.print();
18     constant.print();
19     return 0;
20 }
```

Ausgabe

```
Eine nicht-konstante Methode
Eine konstante Methode
```

HOCHSCHULE ESSLINGEN

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Articify - QUIZ

HOCHSCHULE ESSLINGEN

- Welcher Code ist ausführbar?

1)

```
#include <iostream>
using namespace std;
class A
{
public:
    void print() {
        cout << "Objekt von Klasse A." << endl;
    }
};
int main() {
    const A a;
    a.print();
    return 0;
}
```

2)

```
#include <iostream>
using namespace std;
class A
{
public:
    void print(int &value) const {
        cout << "Objekt von Klasse A mit Wert: " <<
value << endl;
    }
};
int main() {
    const A a;
    const int v = 10;
    a.print(v);
    return 0;
}
```

Articify - QUIZ: AUFLÖSUNG

- Welcher Code ist ausführbar?

1)

```
#include <iostream>
using namespace std;
class A
{
public:
    void print() {
        cout << "Objekt von Klasse A." << endl;
    }
};
int main() {
    const A a;
    a.print();
    // Methode ist nicht für konstante Objekte definiert

    return 0;
}
```

2)

```
#include <iostream>
using namespace std;
class A
{
public:
    void print(int &value) const {
        cout << "Objekt von Klasse A mit Wert: " <<
value << endl;
    }
};
int main() {
    const A a;
    const int v = 10;
    a.print(v);
    // Methode erfordert nicht konstante Eingabe

    return 0;
}
```

HOCHSCHULE ESSLINGEN

1 Klassen und Objekte
2 Statische Datenelemente
3 Statische Funktionen
4 Konstante Datenelemente in Objekten
5 Konstante Objekte
6 Freundfunktionen
7 Freundklassen

FREUNDFUNKTIONEN

- Freundfunktionen einer Klasse sind keine Methoden der Klasse. Sie haben aber die gleichen Zugriffsrechte wie solche.
- Eine Klasse erklärt eine Funktion zur Freundfunktion - und nicht umgekehrt.
- Freundfunktionen können im privaten als auch im öffentlichen Teil der Klasse deklariert werden.
- Freundfunktionen werden nicht wie Instanzmethoden auf Objekten aufgerufen. Man muss ihr ein Objekt der Klasse als Parameter übergeben.

BEISPIEL: FREUNDFUNKTION (1)

```
1  class Student
2  {
3    friend void change_grade(Student &s, float g);
4    friend int get_student_count();
5
6    float _grade;
7    static int _count;
8
9  public:
10   Student()
11   {
12     _grade = 1.3;
13     ++_count;
14   }
15   void print()
16   {
17     cout << "Note: " << _grade << endl;
18   }
19 };
```

BEISPIEL: FREUNDFUNKTION (2)

```
1  // Hier kommen die Freundfunktionen:
2  void change_grade(Student &s, float g) { s._grade = g; }
3  int get_student_count() { return Student::_count; }
4
5  int Student::_count = 0;
6  void main()
7  {
8    cout << get_student_count(); // count == 0
9    Student miller;
10   miller.print();              // grade == 1.3
11   change_grade(miller, 1.0);
12   miller.print();              // grade == 1.0
13   cout << get_student_count(); // count == 1
14 }
```

HOCHSCHULE ESSLINGEN

1 Klassen und Objekte
2 Statische Datenelemente
3 Statische Funktionen
4 Konstante Datenelemente in Objekten
5 Konstante Objekte
6 Freundfunktionen
7 Freundklassen

BEFREUNDETE KLASSEN

```
1  class Fraction
2  {
3      int _numerator, _denominator;
4
5  public:
6      ..... // beispielsweise keine Methoden
7
8      friend class FractionUtilities;
9  };
10
11 class FractionUtilities
12 {
13 public:
14     static int get_numerator(Fraction &f);
15     static void set_numerator(Fraction &f, int n);
16     static void print(Fraction &f);
17 };
18
19 void FractionUtilities::set_numerator(Fraction &f, int n)
20 {
21     f._numerator = n;
22 }
```

BEFREUNDETE KLASSEN

```
1  class Student
2  {
3    friend class Dean;
4    float _grade;
5
6  public:
7  };
8
9  // Hier kommen die Freundesklasse:
10 class Dean {
11   public:
12   void change_student_grade(Student &s, float g) { s._grade = g; }
13 };
14
15 void main()
16 {
17   Student miller;
18   miller.print();                          // grade == 1.3
19   Dean smith;
20   smith.change_student_grade(miller, 1.0);
21   miller.print();                          // grade == 1.0
22 }
```

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



- Welcher Code ist ausführbar?

1)

```
#include<iostream>
using namespace std;
class A {
    friend int get_value(A&);
    int _value;
public:
    A(int value): _value(value) {};
};
int get_value(A& a) {
    return a._value;
};
int main() {
    A a(10);
    cout << get_value(a) << endl;
    return 0;
}
```

2)

```
#include<iostream>
using namespace std;
class A {
    int _value;
public:
    A(int value): _value(value) {};
};
class B {
    friend class A;
public:
    int get_value(A& a) {
        return a._value;
    }
};
int main() {
    A a(10);
    B b;
    cout << b.get_value(a) << endl;
    return 0;
}
```

- Welcher Code ist ausführbar?

1)

```
#include<iostream>
using namespace std;
class A {
    friend int get_value(A&);
    int _value;
public:
    A(int value): _value(value) {};
};
int get_value(A& a) {
    return a._value;
};
int main() {
    A a(10);
    cout << get_value(a) << endl;
    return 0;
}
```

2)

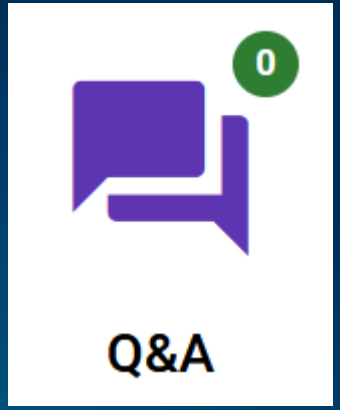
```
#include<iostream>
using namespace std;
class A {
    int _value;
public:
    A(int value): _value(value) {};
};
class B {
    friend class A;
    // B kann sich nicht als Freund von A deklarieren
public:
    int get_value(A& a) {
        return a._value;
    }
};
int main() {
    A a(10);
    B b;
    cout << b.get_value(a) << endl;
    return 0;
}
```

ZUSAMMENFASSUNG

- Statische Klassenelemente (Datenelemente, Funktionen) gehören zu der Klasse und nicht zu den Objekten.
- Klassenvariablen müssen außerhalb der Klasse initialisiert werden.
- Konstante Datenelemente in Objekten werden einmal im Konstruktor (Initialisierungsliste) gesetzt und können dann nicht mehr geändert werden.
- Konstante Instanzfunktionen werden benötigt, wenn man auf konstante Objekte zugreifen will.
- Freundfunktionen, Freundklassen können auch auf private Elemente einer Klasse zugreifen. Die Klasse muss aber seine Freunde selber aussuchen.

HOCHSCHULE ESSLINGEN

FRAGEN?



HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136

HOCHSCHULE ESSLINGEN

KLEINE ÜBUNG

ÜBUNG

Schreiben Sie eine Klasse ClubMember. Jedes Vereinsmitglied (Clubmember) soll eine eindeutige Mitgliedsnummer (_member_id) erhalten. Informationen über die Mitgliedsnummer soll mit der Methode print() ausgegeben werden. Außerdem soll die Anzahl der Mitglieder gespeichert werden und jederzeit abgefragt werden können. Testen sie die Klasse mit einem geeigneten Hauptprogramm.